

实验五 频分多址

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 12 月 17 日

一、实验背景

实验目的：

1. 通过 Matlab 仿真，熟悉误符号率，误比特率及功率谱的概念。
2. 通过对矩形包络下的 BPSK 调制的频分多址（FDMA）进行仿真，深入理解频分多址的概念及实际工作原理。

Matlab 版本：2023b。

二、载波正交 BPSK

1. 实验步骤

- (1) 假定有 $M=4$ 个用户，各自生成 $N=10^5$ 个随机待传信息比特。
- (2) 采用无编码的 BPSK 调制，基带成形脉冲为矩形，持续时间 T 等于符号间隔 $T_s=0.01s$ ，仿真波形的采样间隔取 $\Delta t=10^{-4}s$ 。
- (3) 载波采用 $\sqrt{2}\cos 2\pi f_i t, f_i = (200 + (i - 1) \times 200)Hz, i \in \{1, 2, 3, \dots, M\}$ ，生成对应的发射波形。
- (4) 利用基于滑动窗 FFT 的功率谱近似统计方法绘制发射信号的功率谱，其中 FFT 窗长为 $100T_s$ ，滑动步长为 $80T_s$ 。同时根据功率谱密度计算各用户的功率和总功率。
- (5) 生成高斯噪声，与发送信号相加，得到接收信号。
- (6) 在接收端，使用 $\sqrt{2}\cos 2\pi f_i t, f_i = (200 + (i - 1) \times 200)Hz, i \in \{1, 2, 3, \dots, M\}$ ，对每个用户逐个符号进行相关接收并判决。
- (7) 比较发送信号和接收信号，统计误比特率。
- (8) 编写代码，改变噪声功率谱密度，画出误比特率与 E_b/n_0 之间的关系曲线，并与理论误比特率曲线进行对比。（提示：考虑到功率谱计算相对高的时间复杂度，为提高误比特率统计精度，可在不计算功率谱时适当提高 N ）

2. 代码实现

```
% 通信与网络 实验 5 FDMA
% 载波正交 BPSK
clearvars;
close all;
clc;
rng(2023); %固定随机种子，保证结果可复现

% 脉冲成形波形参数
```

```
T = 0.01; % 周期
V = 1/sqrt(T); % 幅度

% 用户数 (>=2)
N_u = 4;

% 载波参数
delta_fc = 200; % 载波频率间隔
f_lc = 200; % 最低载波频率
f_c = f_lc + (0:N_u-1)*delta_fc; % 载波频率, 满足 f_c(n)*T 为整数

% 最小时间间隔
delta_t = 1e-4;

% 每个符号周期采样次数
num_t = round(T/delta_t);

% 功率谱计算选项: 1 为计算, 0 为不计算
if_ps = 1;
% if_ps = 0;

N_s = 1e5; % 符号个数
% N_s = 2000;
N_b = N_s; % 每用户信息比特数

Es = V^2*T; % 符号能量
Eb = Es; % BPSK 的比特能量

% 功率谱滑动窗参数
win_size = 100; % 滑动窗长与符号周期的比值
win_step = 80; % 滑动窗移动步长与符号周期的比值
win_num = floor((N_s-win_size)/win_step)+1; % 滑动窗个数

% 功率谱频率分辨率
delta_f = 1/T/win_size;

% 功率谱采样频点
f = (0:win_size*num_t-1) * delta_f; % FFT 所得功率谱循环周期为 1/delta_t

EbN0_dB = 0:1:10;
EbN0 = 10.^(EbN0_dB/10);
ber_sim = zeros(length(EbN0), N_u);
ber_theory = qfunc(sqrt(EbN0)); % 理论值可以直接统一计算
```

```
for idx = 1:length(EbN0)
    % 噪声的单边功率谱密度
    % n0 = 0.5;
    n0 = Eb / EbN0(idx) * 2;

    % 仿真时间
    t = (delta_t:delta_t:N_s*T);

    % 成形脉冲
    p_t = V * ones(num_t, 1);

    % 基带脉冲成形
    x0_t = zeros(N_u, N_s*num_t);

    % 发送波形
    x_t = zeros(1, N_s*num_t);

    tic;

    % 随机生成符号序列
    bit_data = randi([0 1], N_u, N_b);

    % 转化为 BPSK 调制后的电平序列
    mod_data = ma_gray_map_real_M2(bit_data);

    % 脉冲成形
    for n_u = 1:N_u
        for n_s = 1:N_s
            x0_t(n_u,num_t*(n_s-1)+1:num_t*n_s) = mod_data(n_u,n_s)*p_t;
        end
    end

    % 逐用户上变频
    for n_u = 1:N_u
        x_t = x_t + x0_t(n_u,:) .* sqrt(2) .* cos(2*pi*f_c(n_u)*t);
    end

    % 功率谱绘图（仅 idx==1 且 if_ps==1 时）
    if if_ps == 1 && idx == 1
        S_X = zeros(1,win_size*num_t); % 功率谱初始化
        for win_id = 1:win_num % 滑动窗编号
            window = (win_id-1)*win_step*num_t+1:(win_id-1)*win_step*num_t+win_size*num_t;
            S_X = S_X + abs(fft(x_t(window))).^2;
        end
    end
end
```

```
S_X = S_X / win_num; % 窗之间求平均
S_X = S_X / (num_t*win_size)^2 / delta_f; % 功率谱密度函数系数

P_total = sum(S_X) * delta_f; % 计算总功率，对 Sx 积分
P_user = zeros(1, N_u);

% 将 Sx 和 f 重新排列到中心
fs = 1/delta_t;
S_X_1 = fftshift(S_X);
f_1 = (-fs/2 : delta_f : fs/2 - delta_f);

for n_u = 1:N_u
    f_ci = f_c(n_u);
    idx_pos = (f_1 >= f_ci - 1.3/T) & (f_1 <= f_ci + 1.3/T); % 正
频率频段
    idx_neg = (f_1 >= -f_ci - 1.3/T) & (f_1 <= -f_ci + 1.3/T); %
负频率频段
    P_user(n_u) = sum(S_X_1(idx_pos | idx_neg)) * delta_f; % 积分
求和
end

fprintf('理论总功率: %.2f W\n', N_u * (1/T));
fprintf('仿真总功率: %.2f W\n', P_total);
for n_u = 1:N_u
    fprintf('用户%d 仿真功率: %.2f W (理论值: %.2f W)\n', n_u,
P_user(n_u), 1/T);
end

figure;
plot(f(f<1/2/delta_t), 10*log10(S_X(f<1/2/delta_t)), 'b',
'LineWidth', 1);
hold on;
plot(f(f>=1/2/delta_t)-1/delta_t, 10*log10(S_X(f>=1/2/delta_t)),
'b', 'LineWidth', 1);
xlabel('频率/Hz');
ylabel('功率谱密度/[dBW/Hz]');
title('功率谱');
xlim([-1/2/delta_t 1/2/delta_t])
ylim(10*log10(max(S_X)*[1E-5 1.2]));
box on;
grid on;
end

% 加性噪声
```

```
noise_power = n0/2/delta_t;  
noise = sqrt(noise_power) * randn(size(t));  
  
% AWGN 信道  
y_t = x_t + noise;  
  
% 等效基带接收波形  
y0_t = zeros(N_u, N_s*num_t);  
  
% 逐用户下变频  
for n_u = 1:N_u  
    y0_t(n_u,:) = y_t .* sqrt(2) .* cos(2*pi*f_c(n_u)*t);  
end  
  
% 初始化接收符号  
y = zeros(N_u, N_s);  
  
% 逐用户最佳接收  
for n_u = 1:N_u  
    for n_s = 1:N_s  
        y(n_u,n_s) = sum(y0_t(n_u, (n_s-1)*num_t+1 : n_s*num_t)) *  
delta_t;  
    end  
end  
  
% 最佳判决结果  
bit_hat = ma_inverse_gray_map_real_M2(y);  
  
% 差错比特  
bit_err = xor(bit_data, bit_hat);  
% 计算各用户的误比特率  
ber = sum(bit_err,2);  
ber = ber/N_b;  
ber_sim(idx,:) = ber;  
  
toc;  
  
% 发送/接收波形绘图（仅 idx==1 时）  
if idx == 1  
    figure;  
    plot((delta_t:delta_t:4*T)', y_t(1:4*num_t), 'LineWidth', 1);  
    hold on;  
    plot((delta_t:delta_t:4*T)', x_t(1:4*num_t), 'LineWidth', 1);  
    xlabel('时间/s');
```

```
        ylabel('幅度/V');
        title('波形图');
        box on;
        grid on;
        legend('接收波形信号','发送波形信号','fontsize',12);
    end
end

% 绘制误比特率与 Eb/n0 之间的关系曲线
figure;
semilogy(EbN0_dB, mean(ber_sim,2), 'o-b', 'LineWidth', 2, 'MarkerSize',
6); hold on;
semilogy(EbN0_dB, ber_theory, '--r', 'LineWidth', 2);
xlabel('E_b/n_0 (dB)');
ylabel('BER');
title('BPSK 传输下误比特率与 E_b/n_0 之间的关系曲线');
legend('仿真 BER','理论 BER');
grid on;

% M=2, 由 bit 序列映射到电平符号
function mod_data = ma_gray_map_real_M2(bit_data)
% 对应的符号序列长度
[N_u, N_s] = size(bit_data);
mod_data = zeros(N_u, N_s);
% 逐用户、逐符号判断映射关系
for n_u = 1:N_u
    for n_s = 1:N_s
        current_bits = bit_data(n_u,n_s);
        if current_bits == 0
            mod_data(n_u,n_s) = -1;
        elseif current_bits == 1
            mod_data(n_u,n_s) = 1;
        end
    end
end
end
end

% M=2, 由电平符号映射到 bit 序列
function demod_bit_data = ma_inverse_gray_map_real_M2(rx_data)
% 符号序列长度
[N_u, N_s] = size(rx_data);
demod_bit_data = zeros(N_u,N_s);
% 逐用户、逐符号判决
for n_u = 1:N_u
```

```

for n_s = 1:N_s
    current_level = rx_data(n_u,n_s);
    if current_level < 0
        demod_bit_data(n_u,n_s) = 0;
    else
        demod_bit_data(n_u,n_s) = 1;
    end
end
end
end

```

3. 实验数据记录及结果分析

(1) 绘制发射信号波形功率谱，根据功率谱密度计算各用户的功率和总功率。

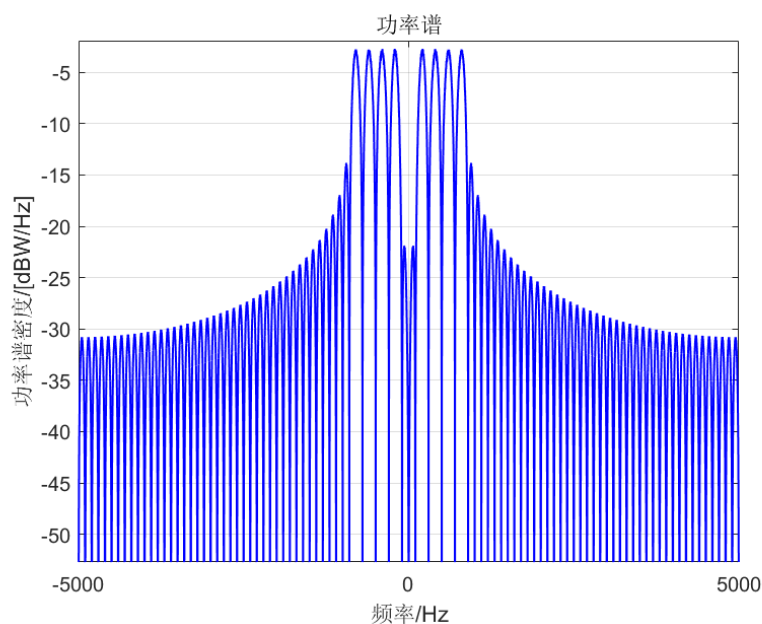


图 1 功率谱

理论推导

对于 BPSK 调制，符号能量 $E_s = V^2 T = \left(\frac{1}{\sqrt{T}}\right)^2 \times T = 1$

符号功率 $P_s = \frac{E_s}{T} = \frac{1}{T}$

载波调制后功率不变，因为载波和接收信号中 $\sqrt{2}$ 的平方与 $\cos^2(2\pi f_c t)$ 的平均值相乘为 1

因此每个用户的发送功率 $P_0 = \frac{1}{T} = \frac{1}{0.01s} = 100W$

总功率 $P = 4P_0 = 400W$

根据功率谱计算功率

对功率谱密度在整个频率范围内积分得到总功率：

```
P_total = sum(S_X) * delta_f; % 计算总功率，对 Sx 积分
```

对功率谱密度在主瓣范围内积分得到各用户功率（经过实验取 $\pm 1.3/T$ 范围内）：

```
% 将 Sx 和 f 重新排列到中心
fs = 1/delta_t;
S_X_1 = fftshift(S_X);
f_1 = (-fs/2 : delta_f : fs/2 - delta_f);
for n_u = 1:N_u
    f_ci = f_c(n_u);
    idx_pos = (f_1 >= f_ci-1.3/T) & (f_1 <= f_ci+1.3/T); % 正频率频段
    idx_neg = (f_1 >= -f_ci-1.3/T) & (f_1 <= -f_ci+1.3/T); % 负频率频段
    P_user(n_u) = sum(S_X_1(idx_pos | idx_neg)) * delta_f; % 积分求和
end
```

```
理论总功率： 400.00 W
仿真总功率： 400.00 W
用户1仿真功率： 93.87 W (理论值： 100.00 W)
用户2仿真功率： 102.41 W (理论值： 100.00 W)
用户3仿真功率： 102.98 W (理论值： 100.00 W)
用户4仿真功率： 99.14 W (理论值： 100.00 W)
```

图 2 各用户功率和总功率

由输出可知，仿真得到的总功率与理论功率比较接近。

- (2) 在多个噪声功率谱密度下，比较发送信号和接收信号，统计误符号率、误比特率。画出误比特率与 E_b/n_0 之间的关系曲线，并与理论误比特率曲线进行对比。

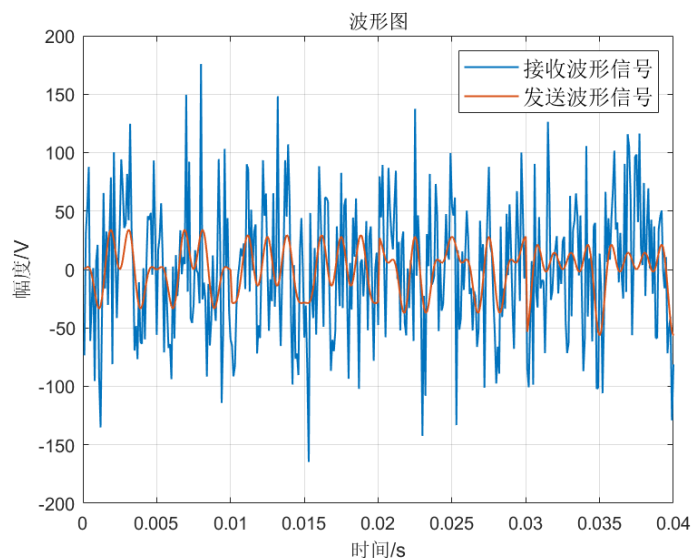


图 3 波形图

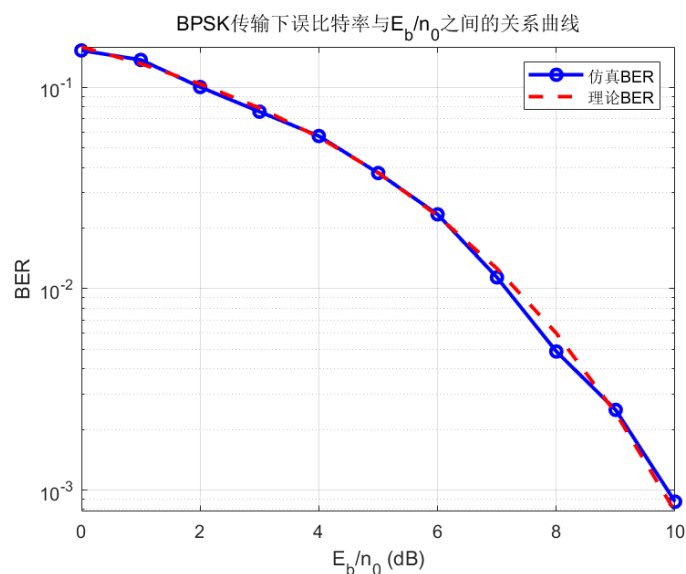


图 4 误比特率与 E_b/n_0 的关系图

理论上，在 BPSK 调制下，误比特率 $P_b = Q\left(\sqrt{\frac{E_b}{n_0/2}}\right)$ ，仿真经过完整的调制解调链路得到误比特率。

由图可知，仿真得到的误比特率曲线与理论值接近。

三、 选做：载波非正交 BPSK

1. 实验步骤

采用方波作为基带成形信号时，若第 i 个载波频率 f_i 与方波信号持续时间 T_s 不满足 $f_i T_s \in \mathbb{N}$ ，如 $(200 + (i - 1) \times \Delta f) \text{ Hz}$ ， Δf 可在集合 $\{30, 50, 80, 100, 130, 150, 180, 200, 230\}$ 中选取时，请仿真 BPSK 调制下的误比特率，绘制收发端信号功率谱函数图像，并理论分析可能导致所观察现象的原因。

2. 代码实现

```
% 通信与网络 实验 5 FDMA
% 载波非正交 BPSK
clearvars;
close all;
clc;
rng(2023); %固定随机种子，保证结果可复现

% 脉冲成形波形参数
T = 0.01; % 周期
V = 1/sqrt(T); % 幅度
```

```
% 用户数 (>=2)
N_u = 4;

% 载波参数
delta_fc = 150; % 载波频率间隔从集合{30,50,80,100,130,150,180,200,230}中选取
f_lc = 200; % 最低载波频率
f_c = f_lc + (0:N_u-1)*delta_fc; % 载波频率, 满足 f_c(n)*T 为整数

% 噪声的单边功率谱密度
n0 = 0.5;

% 最小时间间隔
delta_t = 1e-4;

% 每个符号周期采样次数
num_t = round(T/delta_t);

% 功率谱计算选项: 1 为计算, 0 为不计算
if_ps = 1;
% if_ps = 0;

N_s = 1e5; % 符号个数
% N_s = 2000;
N_b = N_s; % 每用户信息比特数

Es = V^2*T; % 符号能量
Eb = Es; % BPSK 的比特能量

% 功率谱滑动窗参数
win_size = 100; % 滑动窗长与符号周期的比值
win_step = 80; % 滑动窗移动步长与符号周期的比值
win_num = floor((N_s-win_size)/win_step)+1; % 滑动窗个数

% 功率谱频率分辨率
delta_f = 1/T/win_size;

% 功率谱采样频点
f = (0:win_size*num_t-1) * delta_f; % FFT 所得功率谱循环周期为 1/delta_t

EbN0_dB = 0:1:10;
EbN0 = 10.^(EbN0_dB/10);
ber_sim = zeros(length(EbN0), N_u);
ber_theory = qfunc(sqrt(EbN0)); % 理论值可以直接统一计算
```

```
for idx = 1:length(EbN0)
    % 噪声的单边功率谱密度
    % n0 = 0.5;
    n0 = Eb / EbN0(idx) * 2;

    % 仿真时间
    t = (delta_t:delta_t:N_s*T);

    % 成形脉冲
    p_t = V * ones(num_t, 1);

    % 基带脉冲成形
    x0_t = zeros(N_u, N_s*num_t);

    % 发送波形
    x_t = zeros(1, N_s*num_t);

    tic;

    % 随机生成符号序列
    bit_data = randi([0 1], N_u, N_b);

    % 转化为 BPSK 调制后的电平序列
    mod_data = ma_gray_map_real_M2(bit_data);

    % 脉冲成形
    for n_u = 1:N_u
        for n_s = 1:N_s
            x0_t(n_u,num_t*(n_s-1)+1:num_t*n_s) = mod_data(n_u,n_s)*p_t;
        end
    end

    % 逐用户上变频
    for n_u = 1:N_u
        x_t = x_t + x0_t(n_u,:) .* sqrt(2) .* cos(2*pi*f_c(n_u)*t);
    end

    % 功率谱绘图（仅 idx==1 且 if_ps==1 时）
    if if_ps == 1 && idx == 1
        S_X = zeros(1,win_size*num_t); % 功率谱初始化
        for win_id = 1:win_num % 滑动窗编号
            window = (win_id-1)*win_step*num_t+1:(win_id-
1)*win_step*num_t+win_size*num_t;
            S_X = S_X + abs(fft(x_t(window))).^2;
```

```

end
S_X = S_X / win_num; % 窗之间求平均
S_X = S_X / (num_t*win_size)^2 / delta_f; % 功率谱密度函数系数

P_total = sum(S_X) * delta_f; % 计算总功率，对 Sx 积分
P_user = zeros(1, N_u);

% 将 Sx 和 f 重新排列到中心
fs = 1/delta_t;
S_X_1 = fftshift(S_X);
f_1 = (-fs/2 : delta_f : fs/2 - delta_f);

for n_u = 1:N_u
    f_ci = f_c(n_u);
    idx_pos = (f_1 >= f_ci - 1.3/T) & (f_1 <= f_ci + 1.3/T); % 正
频率频段
    idx_neg = (f_1 >= -f_ci - 1.3/T) & (f_1 <= -f_ci + 1.3/T); %
负频率频段
    P_user(n_u) = sum(S_X_1(idx_pos | idx_neg)) * delta_f; % 积分
求和
end

fprintf('理论总功率: %.2f W\n', N_u * (1/T));
fprintf('仿真总功率: %.2f W\n', P_total);
for n_u = 1:N_u
    fprintf('用户%d 仿真功率: %.2f W (理论值: %.2f W)\n', n_u,
P_user(n_u), 1/T);
end

figure;
plot(f(f<1/2/delta_t), 10*log10(S_X(f<1/2/delta_t)), 'b',
'LineWidth', 1);
hold on;
plot(f(f>=1/2/delta_t)-1/delta_t, 10*log10(S_X(f>=1/2/delta_t)),
'b', 'LineWidth', 1);
xlabel('频率/Hz');
ylabel('功率谱密度/[dBW/Hz]');
title(sprintf('功率谱, Δf_c=%dHz', delta_fc));
xlim([-1/2/delta_t 1/2/delta_t])
ylim(10*log10(max(S_X)*[1E-5 1.2]));
box on;
grid on;
end

% 加性噪声

```

```
noise_power = n0/2/delta_t;  
noise = sqrt(noise_power) * randn(size(t));  
  
% AWGN 信道  
y_t = x_t + noise;  
  
% 等效基带接收波形  
y0_t = zeros(N_u, N_s*num_t);  
  
% 逐用户下变频  
for n_u = 1:N_u  
    y0_t(n_u,:) = y_t .* sqrt(2) .* cos(2*pi*f_c(n_u)*t);  
end  
  
% 初始化接收符号  
y = zeros(N_u, N_s);  
  
% 逐用户最佳接收  
for n_u = 1:N_u  
    for n_s = 1:N_s  
        y(n_u,n_s) = sum(y0_t(n_u, (n_s-1)*num_t+1 : n_s*num_t)) *  
delta_t;  
    end  
end  
  
% 最佳判决结果  
bit_hat = ma_inverse_gray_map_real_M2(y);  
% 差错比特  
bit_err = xor(bit_data, bit_hat);  
% 计算各用户的误比特率  
ber = sum(bit_err,2);  
ber = ber/N_b;  
ber_sim(idx,:) = ber;  
  
toc;  
  
% 发送/接收波形绘图（仅 idx==1 时）  
if idx == 1  
    figure;  
    plot((delta_t:delta_t:4*T)', y_t(1:4*num_t), 'LineWidth', 1);  
    hold on;  
    plot((delta_t:delta_t:4*T)', x_t(1:4*num_t), 'LineWidth', 1);  
    xlabel('时间/s');  
    ylabel('幅度/V');
```

```
        title(sprintf('波形图,  $\Delta f_c = %d$ Hz', delta_fc));
        box on;
        grid on;
        legend('接收波形信号', '发送波形信号', 'fontsize', 12);
    end
end

% 绘制误比特率与 Eb/n0 之间的关系曲线
figure;
semilogy(EbN0_dB, mean(ber_sim, 2), 'o-b', 'LineWidth', 2, 'MarkerSize', 6); hold on;
semilogy(EbN0_dB, ber_theory, '--r', 'LineWidth', 2);
xlabel('E_b/n_0 (dB)');
ylabel('BER');
title(sprintf('BPSK 传输下误比特率与 E_b/n_0 之间的关系曲线,  $\Delta f_c = %d$ Hz', delta_fc));
legend('仿真 BER', '理论 BER');
grid on;

% M=2, 由 bit 序列映射到电平符号
function mod_data = ma_gray_map_real_M2(bit_data)
% 对应的符号序列长度
[N_u, N_s] = size(bit_data);
mod_data = zeros(N_u, N_s);
% 逐用户、逐符号判断映射关系
for n_u = 1:N_u
    for n_s = 1:N_s
        current_bits = bit_data(n_u, n_s);
        if current_bits == 0
            mod_data(n_u, n_s) = -1;
        elseif current_bits == 1
            mod_data(n_u, n_s) = 1;
        end
    end
end
end
end

% M=2, 由电平符号映射到 bit 序列
function demod_bit_data = ma_inverse_gray_map_real_M2(rx_data)
% 符号序列长度
[N_u, N_s] = size(rx_data);
demod_bit_data = zeros(N_u, N_s);
% 逐用户、逐符号判决
for n_u = 1:N_u
```

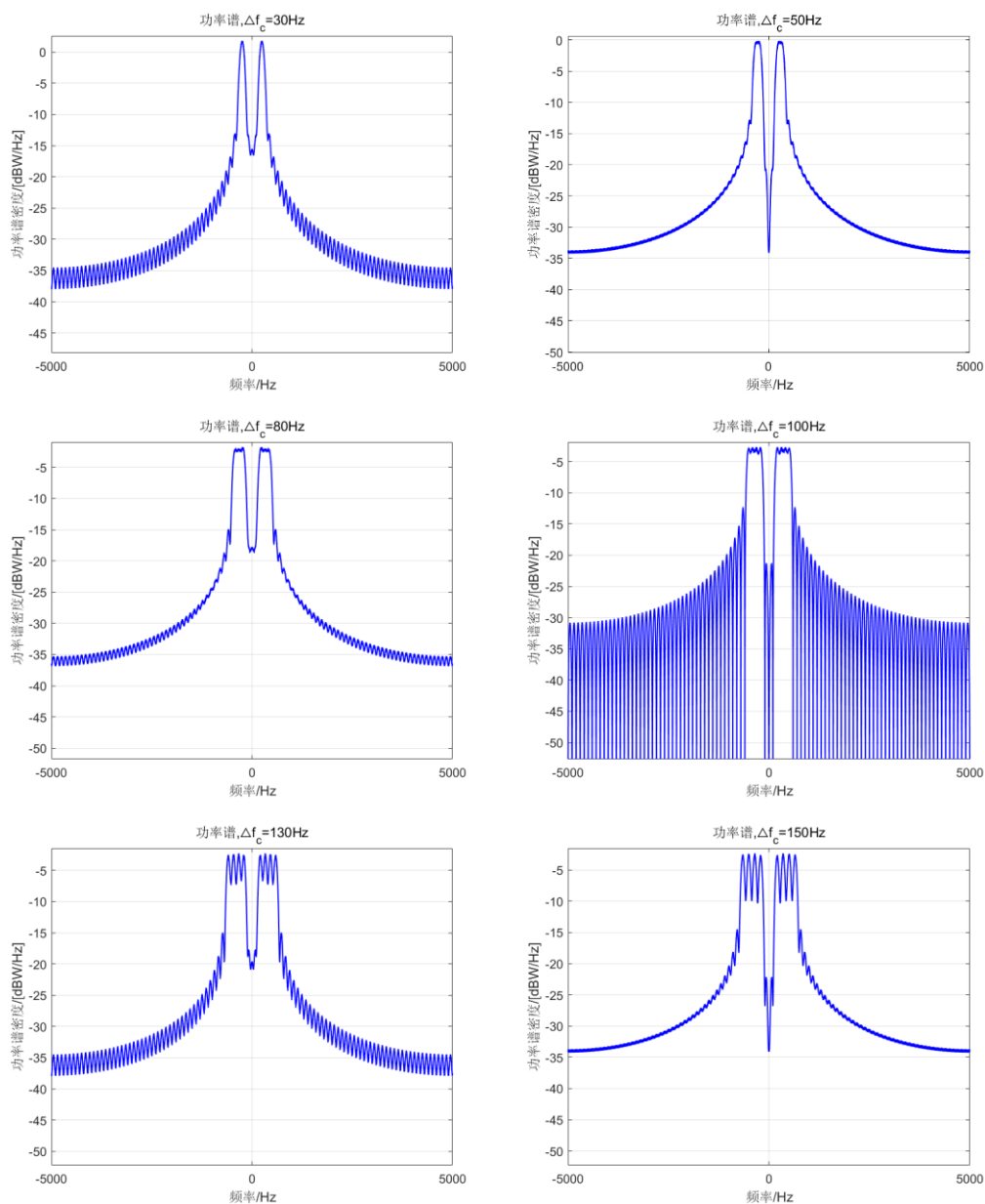
```

for n_s = 1:N_s
    current_level = rx_data(n_u,n_s);
    if current_level < 0
        demod_bit_data(n_u,n_s) = 0;
    else
        demod_bit_data(n_u,n_s) = 1;
    end
end
end
end

```

3. 实验数据记录及结果分析

(1) 绘制发射信号波形功率谱，根据功率谱密度计算各用户的功率和总功率。



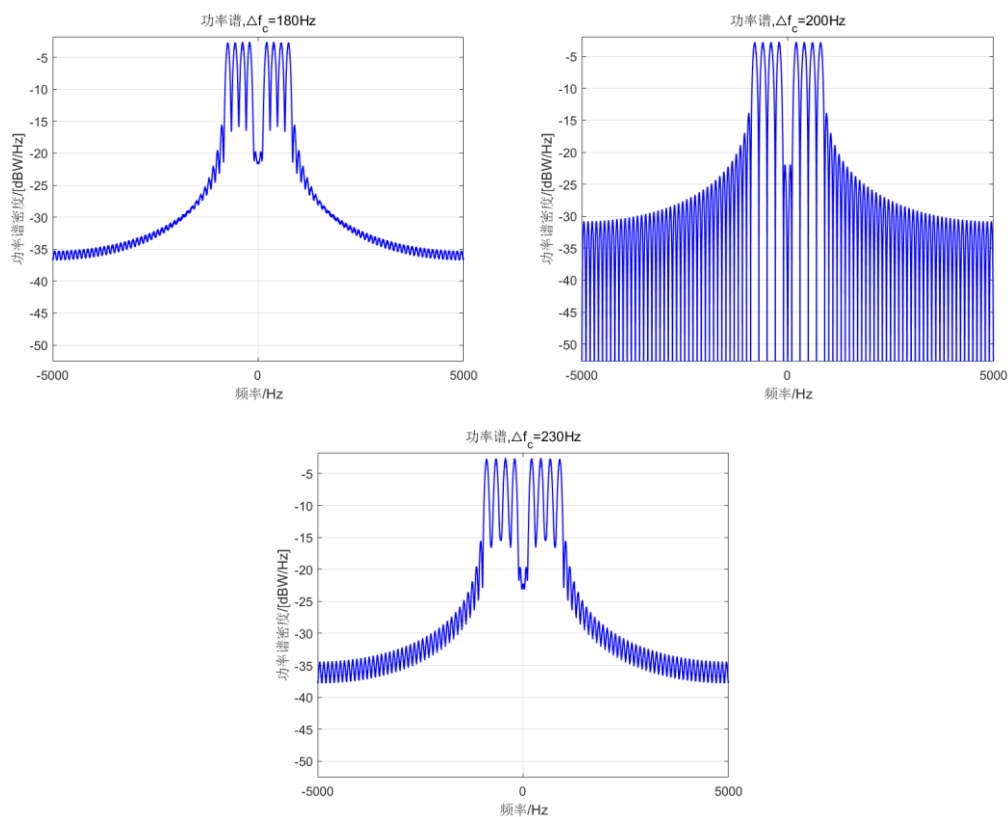
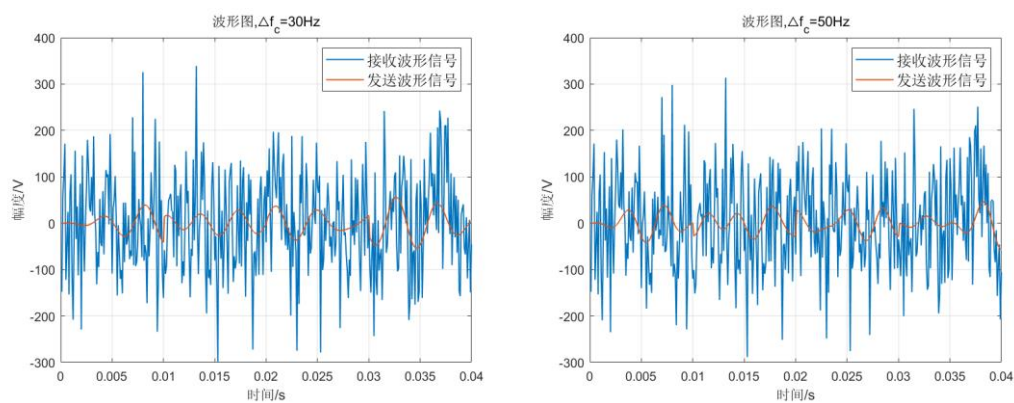


图 5 功率谱

由图可以看出，随着 Δf 增加，功率谱图像中的主瓣宽度增加。此外，在 $\Delta f=100\text{Hz}$ 或 200Hz 时， $f_i T_s \in \mathbb{N}$ ，此时旁瓣的功率谱密度波动范围较大，非正交时旁瓣波动小。

(2) 在多个噪声功率谱密度下，比较发送信号和接收信号，统计误符号率、误比特率。画出误比特率与 E_b/n_0 之间的关系曲线，并与理论误比特率曲线进行对比。



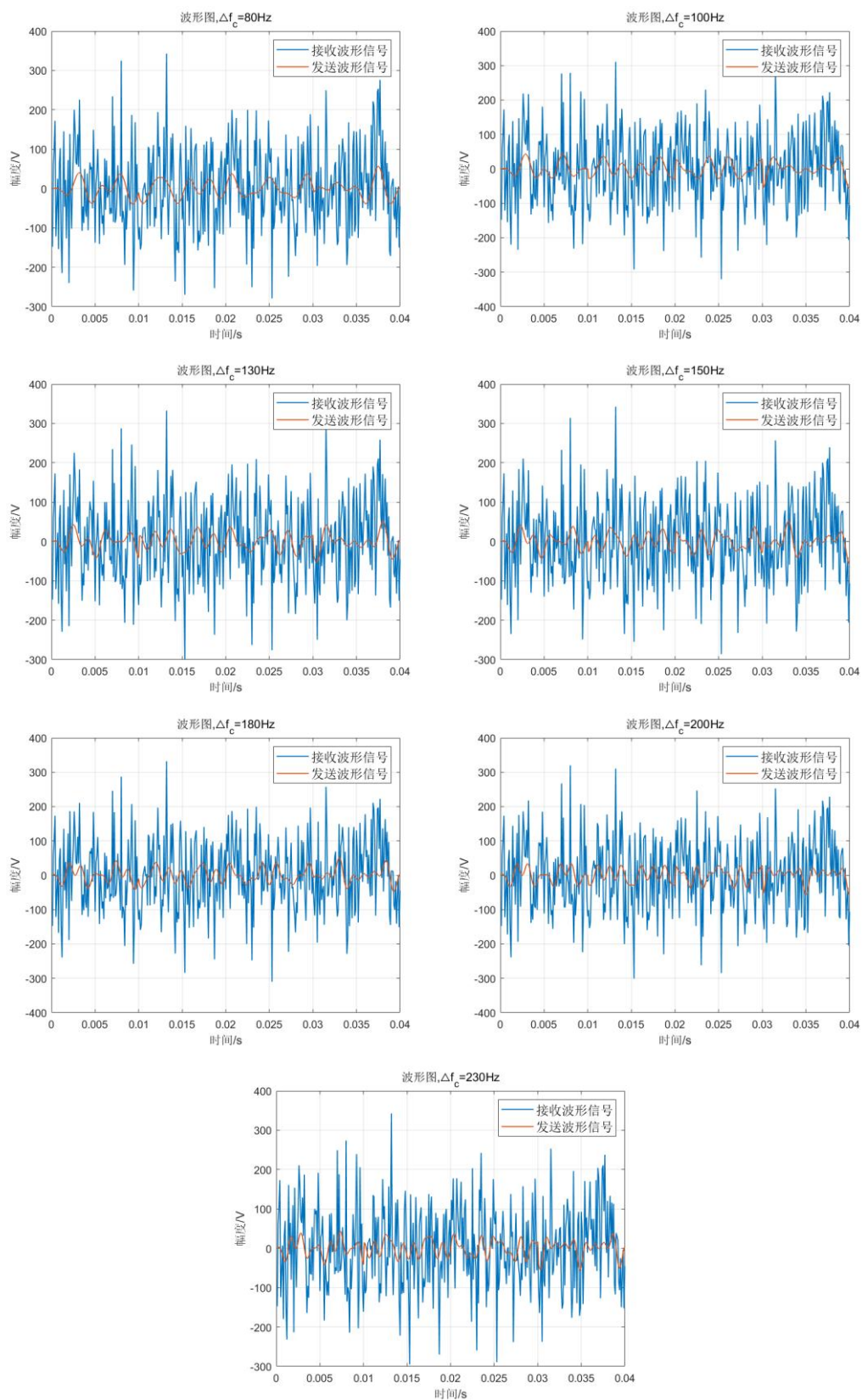
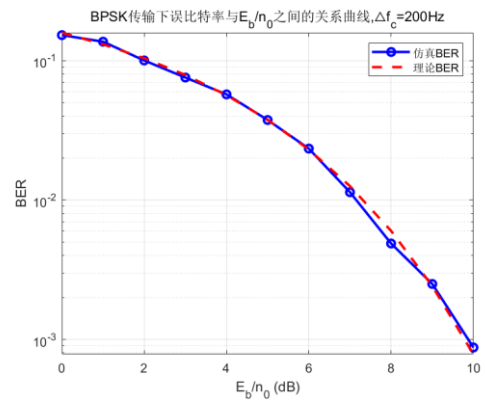
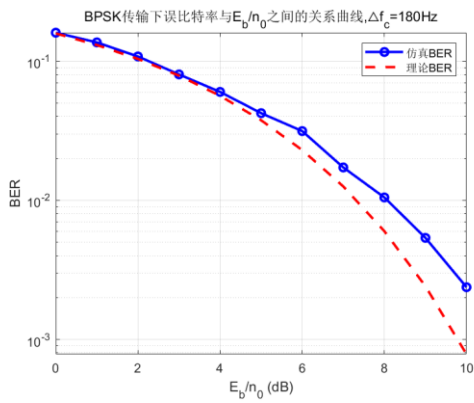
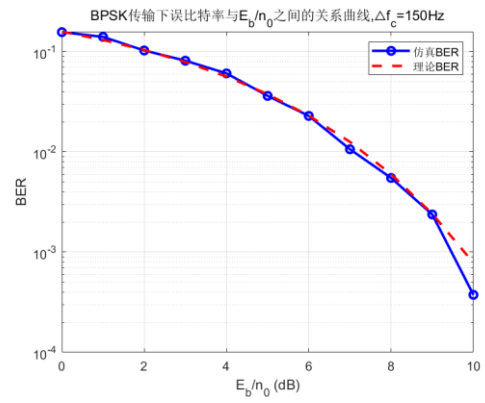
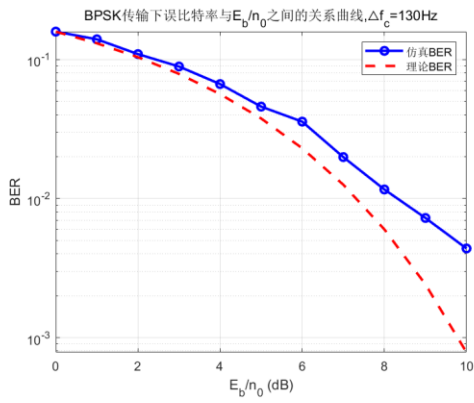
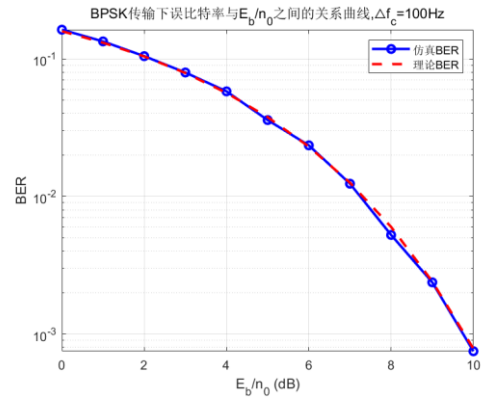
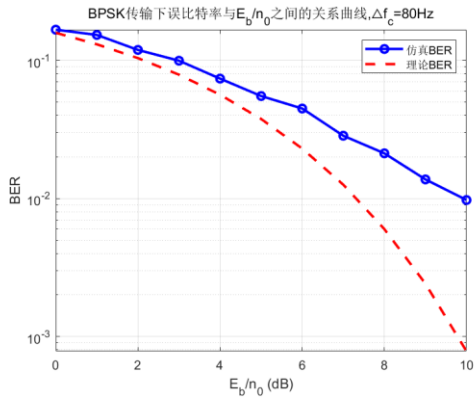
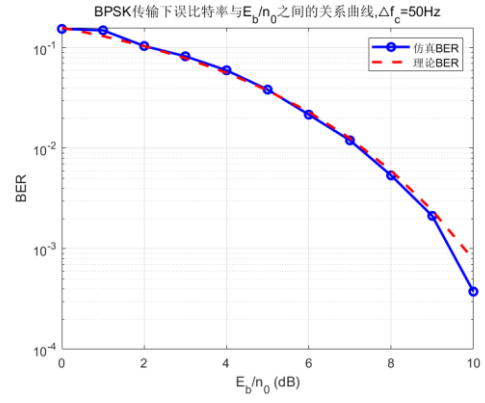
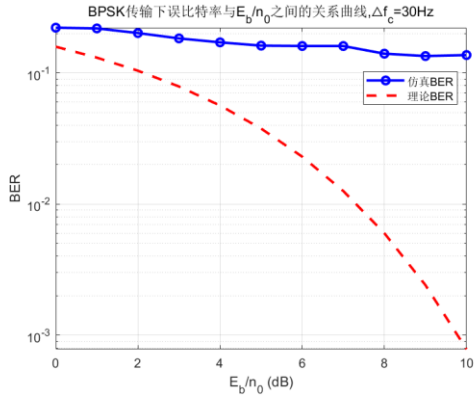
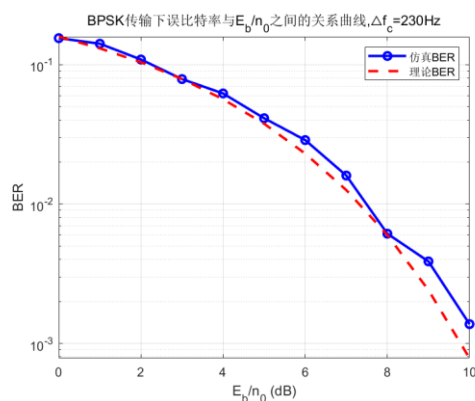


图 6 波形图

由图可知， Δf 的变化对接受波形信号和发送波形信号的影响较小。



图 7 误比特率与 E_b/n_0 的关系图

由图可知，在 $\Delta f=100\text{Hz}$ 或 200Hz 时， $f_i T_s \in \mathbb{N}$ ，此时仿真得到的误比特率曲线与理论值的拟合情况良好；非正交时实验得到的误比特率曲线与理论值偏差较大，实验值大于理论值，在较高信噪比时偏差尤其明显。这是因为非正交时其他用户信号不能被完全滤除，引入的噪声增加，在相同 E_b/n_0 时计算得到的误比特率偏高。

四、 实验总结

本次通信与网络课程实验通过 MATLAB 仿真研究了矩形包络下 BPSK 调制的频分多址 (FDMA)。实验首先绘制了发端信号的功率谱函数图像，并根据功率谱密度计算各用户的功率和总功率。此外，实验还探究了误比特率的变化情况。通过仿真发现，当载波频率满足 $f_i T_s \in \mathbb{N}$ 的正交条件时，各用户信号在接收端可实现完全分离，仿真误比特率曲线与理论值高度吻合；反之，若载波频率间隔不满足正交条件，用户间将产生多址干扰，导致误比特率显著恶化，尤其在较高信噪比时会出现明显的性能损失。

通过本次实验，我基本熟悉了频分多址 (FDMA)、功率谱密度、误比特率等核心概念，也提升了 MATLAB 编程的熟练度。